



安徽大學
Anhui University

《JAVA程序设计》

第二章 标识符和数据类型

🎤 主讲人：黄小平

本章目录



安徽大學
Anhui University

1

JAVA的基本语法单位

2

JAVA编码体例

3

JAVA的基本数据类型

4

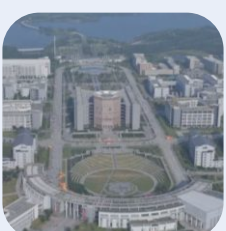
复合数据类型

5

类和对象的初步介绍

6

本章习题解答



2.1 JAVA的基本语法单位



安徽大學
Anhui University

一般来说，一个Java程序的结构，包括以下几个部分：

- package语句：可以没有，或者每一个.java文件只能拥有一个，放在文件开始的位置。
- import语句：可以没有，可以有多个；在package之后，所有类定义之前。
- public型的类定义：每个文件中最多有一个。
- n个类定义，类定义是生成对象的“蓝本”。
- n个接口定义。n=0~N

2.1.1 空白、注释及语句



安徽大學
Anhui University

空白

- 换行符及回车键、空格键、水平定位键（tab）都是空白。Java程序的元素之间可插入任意数量的空白，编译器将忽略掉多余的空白
- 程序中除了加入适当的空白外，还应使用缩进格式，使得同一层语句的起始列位置相同

2.1.1 空白、注释及语句



左右两边哪种书写风格好?

```
class Point {int x,y;Point(int x1,int y1) {x=x1;  
y=y1;  
}  
Point(  
) {this(0,0);}   
void  
moveto(int x1,int y1){  
x=x1;y=y1;  
}}
```

```
class Point {
```

```
    int x, y;                // 点的x轴、y轴坐标
```

```
    Point(int x1, int y1)    // 构造方法  
    {    x = x1;  
        y = y1;  
    }
```

```
    Point()                  // 构造方法  
    {    this( 0, 0);  
    }
```

```
    void moveto(int x1, int y1) // 点移动到 (x1, y1)  
    {    x = x1;  
        y = y1;  
    }
```

```
}
```

2.1.1 空白、注释及语句



安徽大學
Anhui University

注释

- 程序中适当地加入注释，会增加程序的可读性
- 程序中允许加空白的地方就可以写注释。注释不影响程序的执行结果，编译器将忽略注释
- Java中的三种注释形式

// 在一行的注释

/* 一行或多行的注释 */

/** 文档注释 */

2.1.1 空白、注释及语句



安徽大學
Anhui University

注解 → @符号

- ❑ 批注是用于Java语言的本机元数据标记。它们的输入严格与Java语言的其他部分类似，可以通过反映被发现，更容易地让IDE和编译器的编写者管理。批注能够消除样板代码，让源代码的可读性更高，并能提供级别更高的错误检查。从EJB3到JUnit4，哪里都在使用它。
- ❑ **@Override**: 当子类重写父类方法时，子类可以加上这个注解，这可以确保子类确实重写了父类的方法，避免出现低级错误。
- ❑ **@Deprecated**: 当其他程序使用已过时的类，方法时编译器会给出警告。
- ❑ 更多请关注：https://blog.csdn.net/weixin_43748564/article/details/107375850

2.1.1 空白、注释及语句



安徽大學
Anhui University

语句，分号和块

- Java中的语句是最小的执行单位。
- Java各语句间以分号 “;” 分隔。一个语句可写在连续的若干行内。
- 花括号 “{” 和 “}” 包含的一系列语句称为语句块，简称为块。
- 语句块可以嵌套：即语句块中可以含有子语句块。在词法上，块被当作一个语句看待

2.1.2 关键字



abstract	boolean	break	byte	case
catch	char	class	const	continue
do	double	else	extends	false
while	cast	default	final	finally
float	for	future	generic	goto
if	implements	import	inner	instanceof
int	interface	long	native	new
null	operator	outer	package	private
protected	public	rest	return	short
static	super	switch	synchronized	this
throw	throws	transient	true	try
var	void	volatile		

2.1.3 标识符



安徽大學
Anhui University

- ❑ 标识符是以字母、下划线_或美元符\$开头，
由字母、数字、下划线_或美元符\$组成的字符串。
- ❑ 标识符**区分大小写**，长度没有限制
- ❑ 不能是关键字，不能含有其他符号，例如+ - * /等。

在Java语言中，下面变量名命名正确的是（）

- A. #mynam, class, MAX_VALUE, drawImage、
- B. interger、2bQingnian、-abc、A_
- C. ku.gou、iByte、myInt_、myClass
- D. this、thisOne、A+、ThisIsMyFriend、
- E. sizeof、_123、x34y56z78、\$1

2.1.3 标识符



安徽大學
Anhui University

```
1 public class JavaMain {  
2     public static void main(String[] args) {  
3  
4         int sizeof=112;  
5         int 安徽大学=211;  
6         System.out.println("AHU="+安徽大学+",Area"+sizeof);  
7     }  
8 }
```

Java对UTF-8的编码支持，导致在类名、变量名、方法名等可以使用汉字，但是从编码的规范性、维护性角度考虑：不建议使用汉字编写代码！

2.2 JAVA编码体例



安徽大學
Anhui University

Java中的一些命名约定

- 类：类名应为名词，含有大小写，每个字的首字母大写。
 - 例如：
- 接口：接口是一种特殊的类，接口名的命名约定与类名相同。
 - 例如：
- 方法：方法名应是动词，含有大小写，首字母小写，其余各字的首字母大写，尽量不要在方法名中使用下划线。
 - 例如：

2.2 JAVA编码体例



安徽大學
Anhui University

Java中的一些命名约定

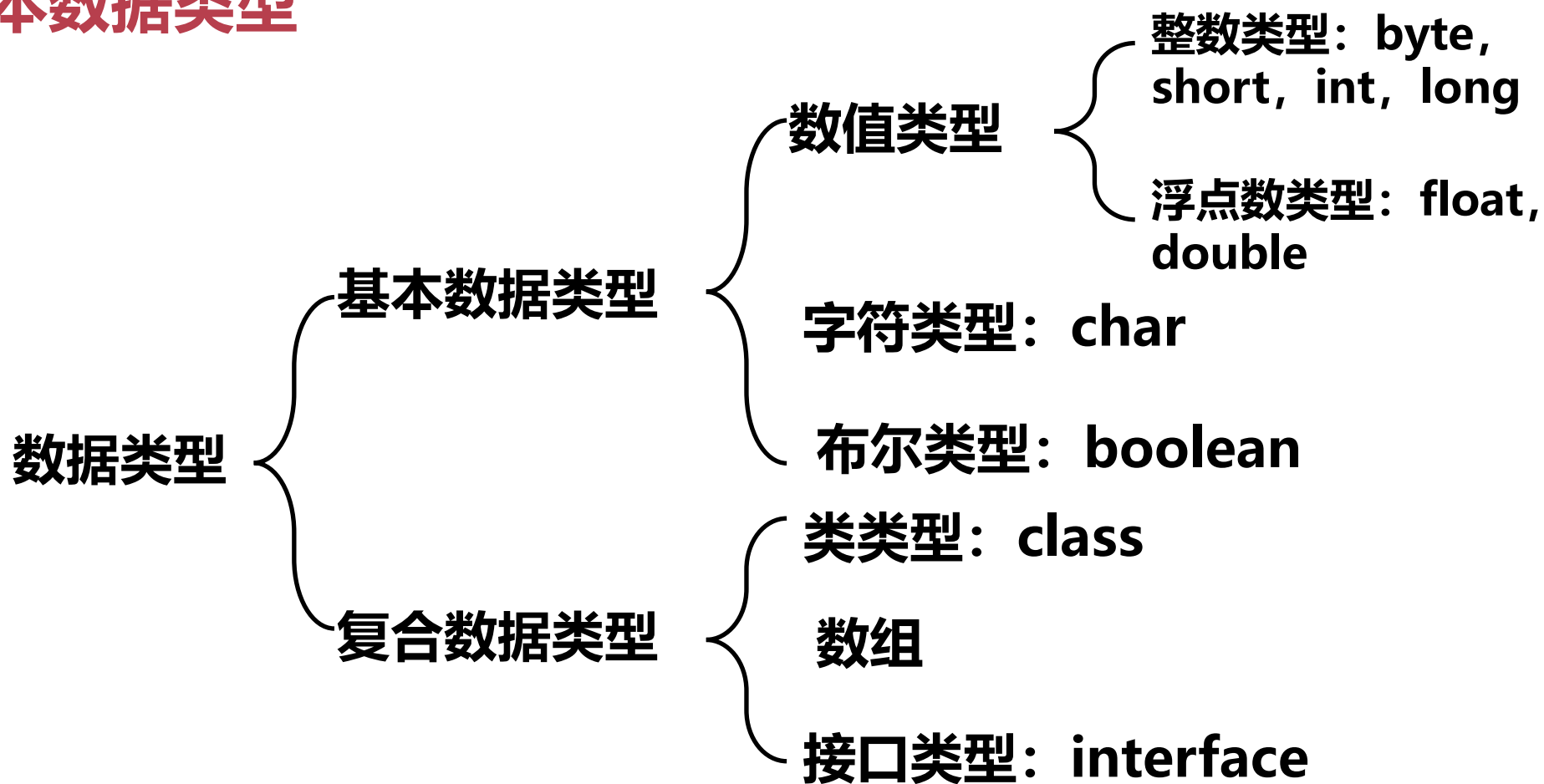
- ❑ 常量：简单类型常量的名字应该全部为大写字母，字与字之间用下划线分隔，对象常量可使用混合大小写。
 - 例如：
- ❑ 变量：所有的实例变量、类变量和全局变量都使用混合大小写，首字符为小写，后面的字首用大写，作为字间的分隔符。变量名中不要使用下划线。
 - 例如：
- ❑ if-else或for结构中一定要用大括号，哪怕只有一行语句。
- ❑ 每行只写一条语句。

2.3 JAVA基本数据类型



安徽大学
Anhui University

基本数据类型



2.3.1 基本数据类型



安徽大學
Anhui University

1. 布尔类型->boolean

- true和false两种状态，在计算机中用8个二进制位来表示。
- 在C++中0表示false, 非0表示ture;而Java不允许这样做，不允许数值类型与布尔类型之间进行转换。

2. 字符类型-> char

- 单个字符用char类型表示，用一个Unicode字符，占用16位无符号二进制位。
- char类型的常量值必须用单引号 ' ' 括起来。
 - 例如: 'a' , ' \t' , '\u????'

2.3.1 基本数据类型



安徽大學
Anhui University

3. 整型->byte, short, int 和 long

Java语言中提供4种的整型量，且都是有符号的。

常用int表示整形量，L表示long型，大小写皆可，但是建议用大写的L。

注意Integer.MAX_VALUE、Integer.MIN_VALUE，Long. MAX_VALUE等

类型	整数长度	字节数	表示范围	举例
byte	8位	1	$-2^7 \sim 2^7 - 1$	byte b=-128
short	16位	2	$-2^{16} \sim 2^{16} - 1$	short s=-32768
int	32位	4	$-2^{32} \sim 2^{32} - 1$	int n=-2147483648
long	64位	8	$-2^{64} \sim 2^{64} - 1$	long l=9223372036854775808L

2.3.1 基本数据类型



安徽大學
Anhui University

4. 浮点型->float和double

float表示单精度的浮点数，double表示双精度的浮点数，且都是有符号的。

类型	整数长度	字节数	表示范围	举例
float	32位	4	1.4e-45f~3.4028235e+38f	float f=5.31
double	64位	8	4.9e-324d~1.79...e+308d	5f 或 5d

注意：Float.MAX_VALUE, Float.MIN_VALUE, Double .MAX_VALUE, Double .MIN_VALUE等的使用。

2.3.2 类型转换



安徽大學
Anhui University

1. 位数少的向位数多的转换

boolean -x-> byte -> short -> char -> int -> long -> float -> double

位数: 8 8 16 16 32 64 32 64

从左到右自动类型转换;

2. 位数多的向位数少的转换

需要用户明确指明, 即强制类型转换, 例如:

```
int i=3;
```

```
byte b=(byte) i;
```

一般地, 高级类型转为低级类型时, 会截断高位内容, 从而导致精度下降或数据溢出。

2.3.3 变量、说明和赋值



安徽大學
Anhui University

程序2-2： 变量使用之前，要先说明或者声明。

```
1 //
2 // 程序 2-2 变量的说明和赋值
3 //
4 public class JavaMain {
5     public static void main(String[] args) {
6
7         int x,y;           // 说明整型变量
8         float z = 3.1414f; // 说明浮点型变量并赋值
9         double w = 3.1415; // 说明双精度型变量并赋值
10        boolean truth = true; // 说明declare and assign boolean
11        boolean false1; // 说明布尔类型变量
12
13        char c;             // 说明字符类型变量
14        c = 'A';            // 给字符类型变量赋值
15        x = 6;
16        y = 1000;           // 给整型变量赋值
17        false1 = 6 > 7; // 给布尔类型变量赋值
18    }
19 }
```

2.3.3 变量、说明和赋值



安徽大學
Anhui University

程序2-3： 为每种基本类型定义一个变量，并为其赋值。

```
1 public class JavaMain {
2     public static void main(String[] args) {
3         // 变量定义（声明）
4         boolean flag;
5         char yeschar;
6         byte finbyte;
7         int intvalue;
8         long longvalue;
9         short shortvalue;
10        float floatvalue;
11        double doublevalue;
12        // 变量赋值
13        flag = true;
14        yeschar = 'y';
15        finbyte = 30;
16        intvalue = -70000;
17        longvalue = 2001;
18        shortvalue = 20000;
19        floatvalue = 9.997E-5f;
20        doublevalue = floatvalue * floatvalue;
```

```
21        // 在控制台输出
22        System.out.println("The values are:");
23        System.out.println("布尔类型变量    flag = " + flag);
24        System.out.println("字符类型变量    yeschar = " + yeschar);
25        System.out.println("字节类型变量    finbyte= " + finbyte);
26        System.out.println("整型变量        intvalue= " + intvalue);
27        System.out.println("长整型变量    longvalue= " + longvalue);
28        System.out.println("短整型变量    shortvalue= " + shortvalue);
29        System.out.println("浮点型变量    floatvalue= " + floatvalue);
30        System.out.println("双精度浮点型变量 doublevalue= " + doublevalue);
31    }
32 }
33
```

Console

<terminated> JavaMain (17) [Java Application] C:\Program Files\Java\jre1.8.0_131\bin\javaw.exe (2021年1月5日 下午1:03:25)

The values are:

布尔类型变量 flag = true

字符类型变量 yeschar = y

字节类型变量 finbyte= 30

整型变量 intvalue= -70000

长整型变量 longvalue= 2001

短整型变量 shortvalue= 20000

浮点型变量 floatvalue= 9.997E-5

双精度浮点型变量 doublevalue= 9.994000294000216E-9

2.4 复合数据类型



安徽大學
Anhui University

早期的程序设计语言把变量看作是孤立的东西。

如果我们在一个程序中需处理日期，则往往说明三个独立的整数分别代表日、月、年。如下所示：

```
int day, month, year;
```

这种方法的不足：如果程序需要处理多个日期，则需要更多的说明。例如要保存两个生日，需如下说明，

```
int myBirthDay,myBirthMonth,myBirthYear;  
int yourBirthday,yourBirthMonth,yourBirthYear;
```

2.4 复合数据类型



安徽大學
Anhui University

缺点：因使用了多个变量而变得混乱，容易出错。同时，又占用了过多的命名空间。更重要的是每个值都是独立的变量。

```
int myBirthDay,myBirthMonth,myBirthYear;  
int yourBirthDay,yourBirthMonth,yourBirthYear;
```

解决办法：

- (1) 提供日期类型。并为这个类型定义了相应的函数，通过调用这些函数就可以得到所需要的结果。
- (2) 定义复合数据类型。复合数据类型为我们提供了更强大的类型定义工具，设计程序时也更加灵活。

2.4 复合数据类型



安徽大學
Anhui University

什么是复合数据类型？

- 定义：用户自定义的新的数据类型称为复合数据类型。
- 在有些语言中，复合数据类型又称作结构类型或记录类型。复合数据类型由程序员在源程序中定义，一旦有了定义，该类型就象其他类型一样使用。
- 对于新定义的复合数据类型，因系统不知道它的具体内容，要由程序员指定其详细的存储结构，这里存储空间的大小不是以字节来衡量，也不是位，而是按已知的其他类型来考虑。
- Java是面向对象的程序设计语言，它为用户提供的复合数据类型就是类、接口和数组。

2.5 类和对象的初步介绍



安徽大學
Anhui University

Java中的面向对象技术

- ❑ 在面向对象程序设计方法之前，软件届广泛流行的是面向过程的设计方式。
该设计方式使用的众多变量名、函数名互不约束，令程序员不堪重负。
- ❑ 面向过程方法设计的程序把处理的主体与处理的方法分开，因此各种成分错综复杂地放在一起，难以理解，易出错，并且难于调试。
- ❑ 随着开发系统的不断扩大，面向过程的方法越来越不能满足使用者的要求
- ❑ OOP技术使得程序结构简单，相互协作容易，更重要的是程序的重用性大大提高了

2.5 类和对象的初步介绍



安徽大學
Anhui University

Java中的面向对象技术

- ❑ 面向对象的方法学，就是使分析、设计和实现一个系统的方法尽可能地接近我们认识一个系统的方法。
- ❑ 面向对象技术主要包含这样几个概念：对象、抽象数据类型、类、类型层次（子类）、继承性、多态性。
- ❑ 面向对象的方法学包括以下三方面：

面向对象的分析（OOA, Object-Oriented Analysis）

面向对象的设计（OOD, Object-Oriented Design）

面向对象的程序设计(OOP, Object-Oriented Program)

2.5.1 JAVA中的面向对象技术



安徽大學
Anhui University

什么是OOP?

□ 面向对象的OOP技术把问题看成是相互作用的事物的集合，用属性来描述事物，而把对它的操作定义为方法。在OOP中，把事物称为对象，把属性称为数据，这样对象就是数据加方法。可以将现实生活中的对象经过抽象，映射为程序中的对象。对象在程序中是通过一种抽象数据类型来描述的，这种抽象数据类型称为类（Class）。

□ OOP中采用了三大技术：

封装

继承

多态

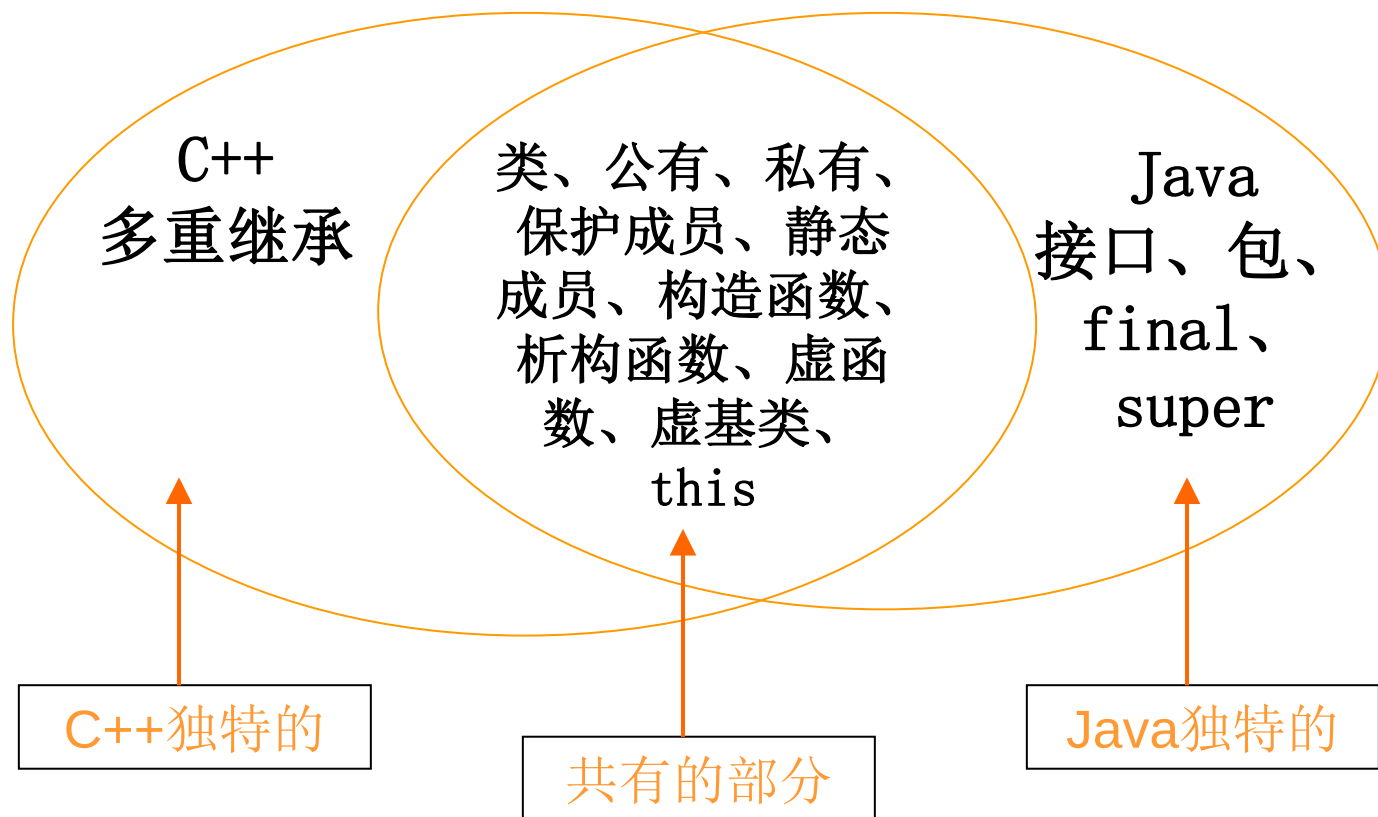
2.5.1 JAVA中的面向对象技术



安徽大学
Anhui University

Java与C++的OOP能力比较

□ Java是完全面向对象的语言，具有完全的OOP能力，它的OOP与C++具有差异

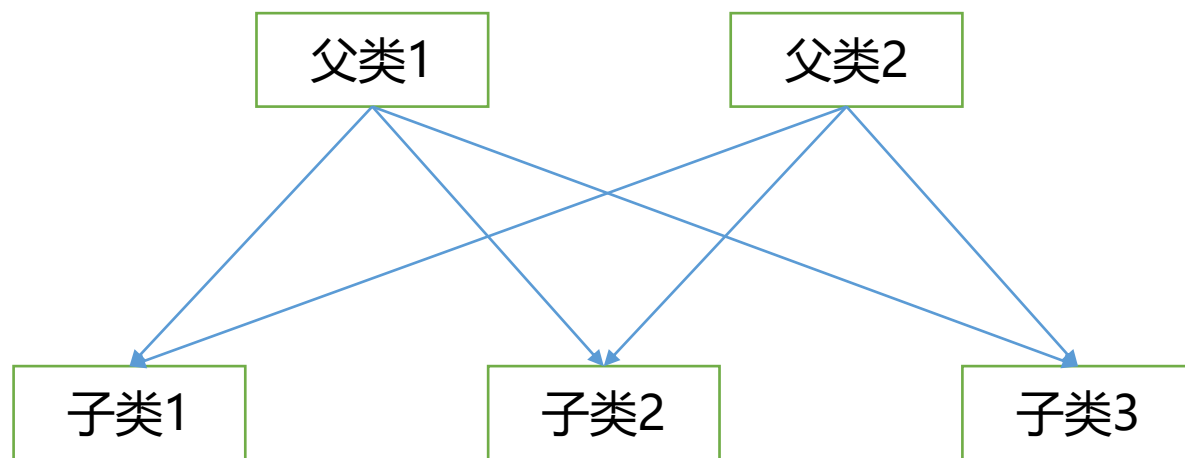


2.5.1 JAVA中的面向对象技术



安徽大学
Anhui University

Java与C++的OOP能力比较



缺点：如果子类的多个父类中有同名的方法和数据，那么容易造成子类实例的混乱。

- ❑ C++中有多重继承的能力，一个类可以有多个父类。
- ❑ 多重集成类似一个网，容易造成

2.5.1 JAVA中的面向对象技术



安徽大學
Anhui University

Java的继承

- ❑ 抛弃了多重继承，每个子类只允许有一个父类，即单重继承。
- ❑ Java中提供接口概念，接口是一种特殊的类，Java多重继承的能力通过接口来实现。
- ❑ Java类层次之上，提出了包的概念，减少了命名冲突，扩大了名字空间。

2.5.2 JAVA中类的定义



Java中类定义的一般格式为：

修饰符 **class** 类名 [**extends** 父类名] {
 类型 成员变量1;
 类型 成员变量2;

 修饰符 类型 成员方法1 (参数列表) {
 类型 局部变量;
 方法体
 }
 修饰符 类型 成员方法2 (参数列表) {
 类型 局部变量;
 方法体
 }

}

```
1 import javax.swing.JFrame;  
2  
3 public class Dog extends JFrame {  
4     public String name; // 狗名  
5     private int age;    // 狗龄  
6     // 构造方法  
7     public Dog(String name, int age) {  
8         this.name=name;  
9         this.age=age;  
10    }  
11    // 方法  
12    private void bark() {  
13        System.out.println("I can bark");  
14    }  
15 }  
16
```

2.5.2 JAVA中类的定义



安徽大學
Anhui University

Java中类定义的一般格式为：

```
修饰符 class 类名 [extends 父类名]{  
    类型 成员变量1;  
    类型 成员变量2;  
    .....  
    修饰符 类型 成员方法1 (参数列表) {  
        类型 局部变量;  
        方法体  
    }  
    修饰符 类型 成员方法2 (参数列表) {  
        类型 局部变量;  
        方法体  
    }  
    .....  
}
```

请仿照左边的语法格式，定义一个人类：

类名： Person

成员变量： name

成员变量： 性别

成员变量： 年龄

成员变量： 身份证号码

成员方法： 说话

成员方法： 走路

成员方法： 唱歌

成员方法： 每个月挣钱5000

2.5.2 JAVA中类的定义



安徽大學
Anhui University

Java中类定义的一般格式为：

```
修饰符 class 类名 [extends 父类名]{  
    类型 成员变量1;  
    类型 成员变量2;  
    .....  
    修饰符 类型 成员方法1 (参数列表) {  
        类型 局部变量;  
        方法体  
    }  
    修饰符 类型 成员方法2 (参数列表) {  
        类型 局部变量;  
        方法体  
    }  
    .....  
}
```

请仿照左边的语法格式，定义一个生日类：

类名：BirthDay

成员变量：年

成员变量：月

成员变量：日

成员方法：设置生日

成员方法：显示生日

2.5.2 JAVA中类的定义



安徽大學
Anhui University

Java中类定义的一般格式为：

```
修饰符 class 类名 [extends 父类名] {  
    类型 成员变量1;  
    类型 成员变量2;  
    .....  
    修饰符 类型 成员方法1 (参数列表) {  
        类型 局部变量;  
        方法体  
    }  
    修饰符 类型 成员方法2 (参数列表) {  
        类型 局部变量;  
        方法体  
    }  
    .....  
}
```

请仿照左边的语法格式，定义一个计算器：
类名： Calculator

成员方法： 加法

成员方法： 减法

成员方法： 乘法

成员方法： 除法

成员方法： 转为2进制

成员方法： 转为8进制

成员方法： 转为16进制

2.5.2 JAVA中类的定义



安徽大學
Anhui University

类定义的几点说明

- Java中的类定义与实现是放在一起保存的，整个 类必须在一个文件中，因此有时源文件会很大。
- Java源文件名必须根据文件中的公有类名来定义，并且要区分大小写。
- 类定义中可以指明父类，也可以不指明。若没有指明从哪个类派生而来，则表明是从缺省的父类Object派生而来。Object是Java中所有类的父类。Java中除Object之外的所有类均有一个且只有一个父类。Object是唯一没有父类的类。
- class定义的大括号之后没有分隔符 “;”

2.5.3 JAVA与OOP有关的关键字



1. 限定访问权限的修饰符

□ 分别为：public、private、protected；不写默认为friendly。

类型	无修饰符	private	protected	public
同一类	是	是	是	是
同一包中的子类	是	否	是	是
同一包中的非子类	是	否	是	是
不同包中的子类	否	否	是	是
不同包中的非子类	否	否	否	是

2.5.3 JAVA与OOP有关的关键字



2. 存储方式修饰符

- static: 既可修饰数据成员，又可以修饰成员方法。
- 类中定义了公有静态变量，则其相当于全局变量。

```
16
17 class Count{
18     private int serialNumber;
19     // 静态变量的定义
20     private static int counter=0;
21     public static int age=0;
22     public Count() {
23         counter++;
24         serialNumber=counter;
25     }
26 }
```

```
27
28 // 公有静态变量的使用
29 class OtherClass{
30     public void method() {
31         // 可以直接通过类名来访问
32         int x= Count.age;
33     }
34 }
```

2.5.3 JAVA与OOP有关的关键字



3. 与继承有关的关键字

- 继承是指特殊类对象共享一般类对象的状态及行为。子类从父类继承的内容包括属性和方法；
- Java语言中，与继承有关的关键字主要有：
 - ✓ final：用final修饰的类不能再派生子类，它已到达类层次中的最低层。
 - ✓ abstract：用abstract可以修饰类或成员方法，表明被修饰的成分是抽象的。

2.5.3 JAVA与OOP有关的关键字



3. 与继承有关的关键字

```
38 // 例2-10 抽象类示例
39 abstract class Shape{
40     abstract void draw();
41     Point position;
42     Shape(Point p) {
43         position=p;
44     }
45 }
46
47 // 派生子类1
48 abstract class Round extends Shape{
49     Round(Point p) {
50         super(p);
51     }
52     final double pi=3.14;
53     abstract void draw();
54     abstract double area();
55 }
```

```
56 // 派生子类2
57 class Circle extends Round{
58     int radius;
59     void draw() {
60         System.out.println("draw()");
61     }
62     double area() {
63         return pi*radius*radius;
64     }
65     Circle(Point p,int radius) {
66         super(p);
67         this.radius=radius;
68     }
69 }
```

2.5.3 JAVA与OOP有关的关键字



安徽大學
Anhui University

4. this和super

□ this指代本类，super指代父类。

2.5.4 类定义示例



安徽大學
Anhui University

用户可以使用类构造各种类型

例2-11

```
public class Date{  
    int day;  
    int month;  
    int year;  
}
```

在C语言中是不是结构体？

例2-12

```
class Point{  
    int x,y;  
    Point(int x1, int y1) {  
        x=x1;  
        y=y1;  
    }  
    Point() {  
        this(0, 0);  
    }  
    void moveTo(int x1, int y1){  
        x=x1;  
        y=y1;  
    }  
}
```

例2-13

```
class Point3d extends Point{  
    int z;  
    public Point3d(int x,int y,int z){  
        super(x,y);  
        this.z=z;  
    }  
    public Point3d(){  
        this(0, 0, 0);  
    }  
}
```


2.5.4 类定义示例



安徽大學
Anhui University

请编写教材P37页： 程序2-4，并运行结果。



2.5.5 创建一个对象

- ❑ 对于基本类型的变量，可以用boolean、byte、short、char、int和long等，相应地系统要为其分配内存。
- ❑ 类的定义过程实际上相当于制作“模子”，创建一个类类型变量称为创建一个对象，就像拿着模子复制了一个副本。
- ❑ 使用类类型创建变量，系统并不分配内存，仅在内存中建立一个引用，并置初值为null，表示不指向任何内存空间。
- ❑ 仅当用关键字new时，才为对象申请内存空间。



2.5.5 创建一个对象

1. 对象引用

□ 声明一个引用的变量的语法格式如下：

类名 变量名;

例如：Point p; Person jack; Customer bob;

Java系统在内存中为实例分配相应的空间后会将存储地址返回，称此存储地址为对象的应用（reference），也可以称为引用变量。

2.5.5 创建一个对象



2. 对象实例化

- 说明一个引用变量仅仅是预定了变量的存储空间，此时并没有变量生产，也没有相应空间给实例使用，必须实例化之后，才有真正的实例出现。
- 创建对象实例的格式如下：

变量名 = new 类名 (参数列表);

例如：Point p;

Person jack;

Customer bob;

p=new Point(0,0);

jack=new Person();

bob=new Customer()

实例化过程实际上是为该对象分配内存。当一个对象实例不被任何变量引用时，Java会自动启动垃圾回收线程，回收它的内存空间。

2.5.5 创建一个对象



安徽大学
Anhui University

3. 对象使用

□ 使用对象中的数据和方法的格式如下：

对象引用.成员数据

对象引用.成员方法(参数列表);

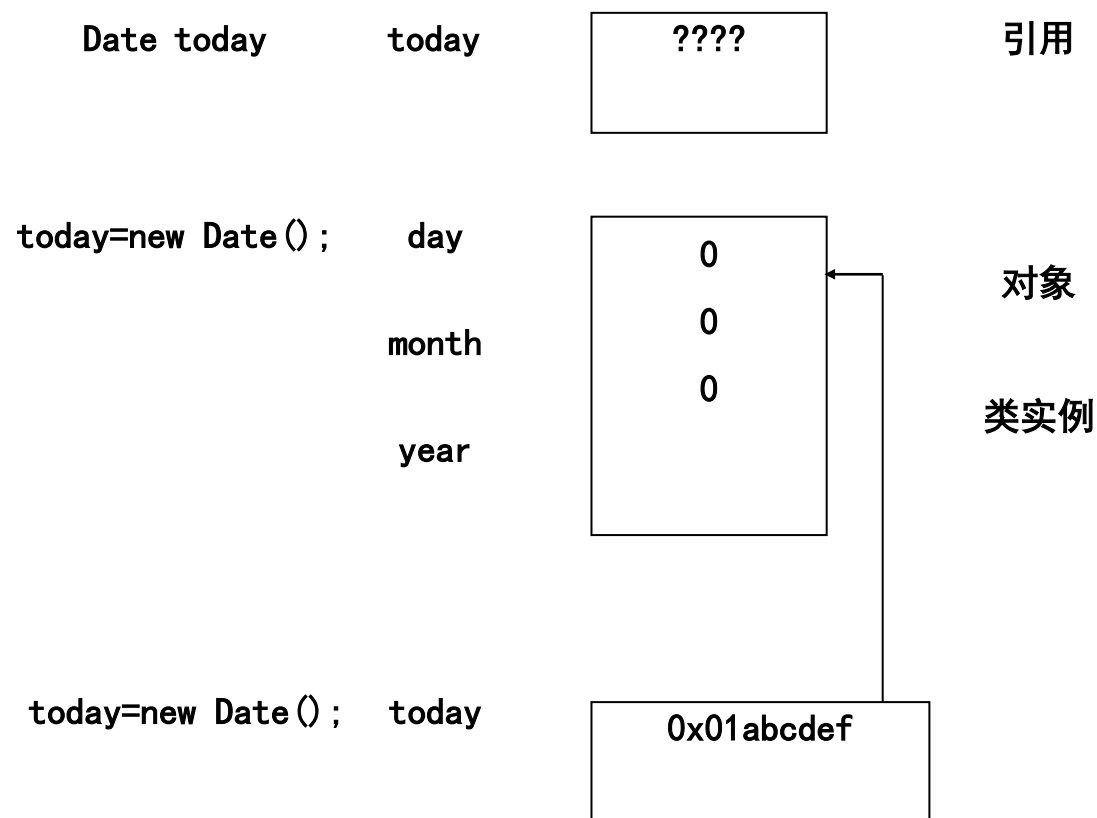
例如：

```
Point p=new Point(1,1);
```

```
p.x=p.y=20;
```

```
system.out.print( "p.x=" + p.x)
```

```
system.out.print( "p.y=" + p.y)
```



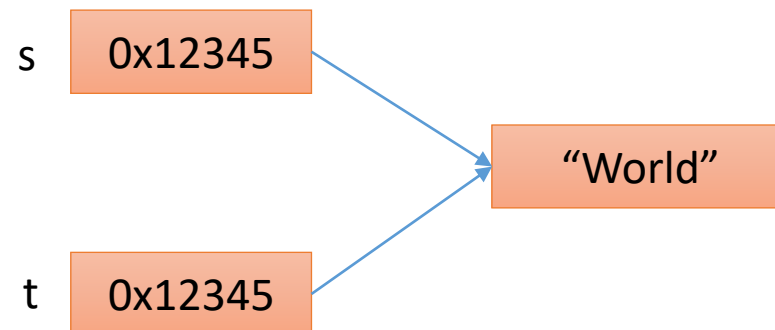
2.5.6 引用变量的赋值



Java把说明为class类型的变量看作是引用，由此，赋值语句的含义有了重大改变。
请看下面的对比：

```
int x=7;  
int y=x;  
x=5;  
y=?
```

```
String s= "Hello" ;  
String t =s;  
s= "World" ;  
t=?
```



结论：基本数据类型，变量的引用互相独立；

class数据类型，变量的引用相互影响； 与C/C++指针有类似之处。

<https://www.zhihu.com/question/402454892/answer/1293132943>

2.5.7 默认初始化和NULL引用值



当执行new为一个对象分配内存时，Java自动初始化所分配的内存空间。

- 对于数值变量，赋初值0;
- 对于布尔变量，赋初值为false;
- 对于引用，使用特殊的值null;

在Java中，null值表示引用不指向任何对象。运行过程中系统发现使用了这样一个引用时，可以停止访问，不会给系统带来任何危险。

```
Customer bob;  
system.out.println(bob)
```

```
Customer bob=null;  
system.out.println(bob)
```

2.5.7 默认初始化和NULL引用值



安徽大學
Anhui University

Java中规定：

- 任何变量使用之前，必须对变量赋值；
- null 字母全部小写！

本章讲课结束

谢 谢！